DATABASE ARCHITECTURE SPECIFICATION

CareerForge AI: Supabase PostgreSQL Database, Schemas, & Data Pipeline

Comprehensive Guide to Database Design, Relational Schemas, JSONB State Models, and Cloud Sync

Database Engine:	PostgreSQL 15 (Managed via Supabase Cloud)
Architecture Style:	Hybrid Relational (Auth) + Document JSONB (Student Progress State)
Core Tables:	1. auth.users (Managed Authentication) 2. public.student_states (Live Application Data)
Security Model:	Row Level Security (RLS) with JWT-based User Isolation
Connection Protocol:	HTTPS REST PostgREST API + Realtime WebSockets
Document Target:	Technical Viva Voce, Academic Examination & Engineering Review

Executive Summary: CareerForge AI utilizes a cloud-native PostgreSQL database hosted on Supabase. By adopting a hybrid architecture—using standard relational tables for authentication and high-performance JSONB binary storage for application state—the database achieves atomic data persistence, zero-overhead schema migrations, sub-millisecond query execution, and real-time faculty synchronization without redundant database joins.

1. Database Architecture & Design Rationale

CareerForge AI's persistence layer is engineered to satisfy two distinct demands: enterprise-grade identity security for authentication, and ultra-flexible, real-time reactive state synchronization for student career journeys. To accomplish this, the database employs a dual-schema structure:

- **Schema 1: auth (Authentication Schema):** An isolated, hardened PostgreSQL schema managed by Supabase Auth. Handles student email verification, bcrypt password hashing, session expiration, and cryptographic JWT issuance.
- **Schema 2: public (Application Schema):** The institutional data schema where the student_states table resides. Stores all academic records, readiness scores, resume attempts, interview metrics, and faculty endorsements.

Why Supabase & PostgreSQL 15 Over Traditional MySQL or MongoDB?

Feature	PostgreSQL 15 (Supabase)	Legacy MySQL / MongoDB Alternative
ACID Compliance	Full transactional guarantees across state writes.	MongoDB requires custom multi-document transactions.
Hybrid JSONB Storage	Binary JSON indexing (GIN/BTREE) with instant sub-property lookups.	MySQL JSON has higher overhead; MongoDB lacks relational joins.
Row Level Security (RLS)	Postgres policies execute directly at SQL engine level based on JWT.	Requires heavy server-side middleware and custom route checks.
Real-time Replication	Native PostgreSQL WAL (Write-Ahead Logging) over WebSockets.	Requires separate socket servers (Socket.io) or Redis pub/sub.

2. Complete Table Data Dictionary

Table 1: auth.users (Managed Authentication Entity)

Column Name	Data Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique cryptographic identifier generated per student.
email	VARCHAR(255)	UNIQUE, NOT NULL	Student institutional or personal email address.
encrypted_password	VARCHAR	NOT NULL	Bcrypt salted password hash; plain text is never stored.
email_confirmed_at	TIMESTAMPTZ	NULLABLE	Verification timestamp once student verifies account.
created_at	TIMESTAMPTZ	DEFAULT now()	Account creation audit timestamp.
last_sign_in_at	TIMESTAMPTZ	NULLABLE	Last active login event recorded for telemetry.

Table 2: public.student_states (Core Application State Table)

Column Name	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, FK -> auth.users.id	1-to-1 foreign key link to authenticated student record. Cascades on delete.
student_email	TEXT	NOT NULL, INDEXED	Indexed student email for fast administrative lookups.
state_data	JSONB	NOT NULL, DEFAULT '{}'	Binary JSON document containing the complete reactive application state tree.
updated_at	TIMESTAMPTZ	DEFAULT now()	Auto-updating timestamp tracking every state sync event.

3. Anatomy of the state_data (JSONB) Document

The core innovation in CareerForge AI's database architecture is the structured state_data JSONB object. Instead of fragmenting progress into 15+ normalized relational tables (which requires complex SQL multi-joins), the student's entire reactive state tree is stored atomically in binary JSON:

Property Path	Type	Example Value	Purpose / Screen Usage
profile.name	string	"Aarav Mehta"	Full student name displayed across portals and on certificate.
profile.enrollment	string	"21ENG1042"	College enrollment/roll number used for faculty audits.
profile.dept	string	"Computer Science"	Academic department for cohort grouping in Admin table.
profile.year	string	"4th Year"	Graduation standing for placement eligibility checks.
domain	string	"aiml"	Chosen target domain ID (1 of 25 tracks) locking visualizer & roadmap.
readiness	number	84	Composite 0–100 score powering dashboard circular gauge.
resume.current	number	78	Most recent ATS score calculated on resume analysis.
resume.attempts	number	3	Count of resume draft optimization attempts.
tech.current	number	85	Latest score from Gemini AI Technical Interview practice.
hr.current	number	90	Latest score from Gemini AI HR Behavioral Interview practice.
mockAIFeedback	object	{relevance:80, communication:75...}	Exact 4-tier score breakdown & coaching tip from Gemini Flash.
points	number	1450	Gamification XP points accumulated across practice activities.
streak	number	5	Consecutive days student practiced on the platform.
badges	array	["b_first_quiz", "b_mock_ace"]	Unlocked achievement badges displayed in Rewards Vault.
cert.status	string	"Approved"	Endorsement state: 'Not Eligible', 'Pending', 'Approved', 'Rejected'.
cert.hash	string	"CF-2026-88A9F"	Cryptographic credential verification hash on final certificate.

4. Data Lifecycle: CRUD & Synchronization Pipeline

How the Client Interacts with PostgreSQL Step-by-Step:

- **1. CREATE (Student Registration):** Student calls supabase.auth.signUp(). Upon successful creation, the app initializes an empty state structure and executes an upsert() to insert the initial row into student_states.
- **2. READ (Student Sign-In):** After authenticating via signInWithPassword(), the app runs: supabase.from('student_states').select('*').eq('id', user.id).single(), restoring the student's progress instantly.
- **3. UPDATE (Continuous State Autosave):** Whenever an interview is finished, quiz submitted, or resume analyzed, saveState() triggers. It saves immediately to localStorage (for offline speed) and performs an asynchronous upsert() to PostgreSQL.
- **4. ADMIN QUERY (Faculty Oversight):** When an administrator opens admin.html, admin.js queries: supabase.from('student_states').select('*').order('updated_at', { ascending: false }). Renders live students with zero dummy data.
- **5. GOVERNANCE UPDATE (Faculty Endorsement):** When faculty clicks 'Approve' on a certificate, the admin script updates only the state_data->cert->status field in Supabase. The change reflects on the student's screen in real-time.

5. Database Security & Top 10 Viva Voce Questions

Row Level Security (RLS) Policies Implemented:

- **Student Privacy Isolation:** An RLS policy on student_states ensures that a standard student user can only read and write rows where auth.uid() = id. They cannot view or tamper with any peer's scores.
- **Faculty Admin Oversight:** Administrative service role tokens or institutional faculty roles are authorized to perform full-table select queries and update certificate endorsement statuses.

Q1: Why did you choose a JSONB column inside PostgreSQL instead of 15 normalized tables?

Answer: In modern Single Page Applications (SPAs), the client manages an in-memory reactive state tree. Using JSONB allows atomic, single-trip read/write operations without multi-table locks or expensive SQL joins. It also provides future-proof flexibility: adding new badges, interview categories, or AI feedback fields requires zero ALTER TABLE schema migrations.

Q2: Why did the 'Display Name' appear empty in the Supabase Auth dashboard?

Answer: Supabase auth.users strictly handles authentication credentials (email & password hash). We follow professional separation of concerns: full collegiate profile data (Name, Roll Number, Department, Year) is collected on the Profile Setup screen and stored in our dedicated public.student_states application table.

Q3: What ensures the database doesn't get flooded with duplicate rows per student?

Answer: The id column in public.student_states is defined as the PRIMARY KEY and is a FOREIGN KEY linked to auth.users.id. We use PostgreSQL's UPSERT pattern (INSERT ... ON CONFLICT (id) DO UPDATE), guaranteeing exactly one row per registered student.

Q4: How does the system handle internet disconnection while practicing?

Answer: We implemented dual-layer local caching. saveState() writes first to browser localStorage. If the network drops, the student continues practicing seamlessly. When the connection re-establishes, the latest state is synchronized back to Supabase.

Q5: How does the Admin Portal get real student records with zero dummy data?

Answer: admin.js imports the Supabase client and queries public.student_states directly. It loops through the returned rows and dynamically extracts state_data.profile.name, readiness, and cert.status to populate the table. All dummy seed data arrays were permanently removed.

Q6: How are passwords secured in your database?

Answer: Passwords never reach the database in plain text. Supabase Auth processes passwords through bcrypt with an adaptive salt work factor before writing to auth.users.encrypted_password.

Q7: What is the benefit of PostgreSQL JSONB over standard JSON text?

Answer: Standard JSON stores plain text which requires re-parsing on every query. JSONB stores data in a parsed binary format, supporting GIN (Generalized Inverted Index) indexing, faster sub-attribute lookups, and significantly lower CPU utilization.

Q8: How does the faculty endorsement update sync back to the student?

Answer: When faculty approves a certificate in admin.html, it executes an update on student_states. When the student portal accesses S.cert or reloads the dashboard, it reads the updated 'Approved' status and renders the official credential seal.

Q9: What is Row Level Security (RLS)?

Answer: RLS is a PostgreSQL security feature that evaluates SQL filter predicates at the database engine level for each incoming request based on the caller's JWT token, preventing unauthorized users from accessing other rows even if they bypass the frontend UI.

Q10: What indexes are applied on the student_states table?

Answer: We have a primary B-Tree index on id (UUID), a secondary B-Tree index on student_email for fast searches, and a timestamp index on updated_at for ordering recent cohort activity.